

12 Days Guideline roadmap

The 8-Member Team Breakdown

- **Pair A (GIS & Remote Sensing Leads - 2 Members):** Handlers of satellite imagery, coordinate mapping, and spatial data extraction.
- **Pair B (Backend & Analytics Leads - 2 Members):** Keepers of the server, API endpoints, data fusion calculations, and the simplified prediction engine.
- **Pair C (Frontend & Map UI Leads - 2 Members):** Architects of the dashboard layout, Leaflet.js interactive maps, and visual charts.
- **Pair D (Project Managers & Technical Writers - 2 Members):** Responsible for data synthesis, tracking daily progress, and assembling the final report/PPT.

Day-by-Day Execution Plan

Phase 1: Localizing to Mirpur 12 & Environment Setup (Days 1–3)

Goal: Isolate the geographic scope to Mirpur 12 and launch your development framework.

Day 1: Defining Scope & Boilerplate Setup

- **Pair A:** Map out the exact bounding box coordinates for Mirpur 12 (approx. Latitude: 23.8200 to 23.8300, Longitude: 90.3600 to 90.3700) and locate this area on Google Earth Engine or USGS Earth Explorer.
- **Pair B:** Set up the backend framework (Flask or Django) and configure the local server routing. Create a base API endpoint `/api/heat-data?region=mirpur12`.
- **Pair C:** Set up the frontend workspace (HTML/CSS/Bootstrap or React). Build a placeholder sidebar with a region dropdown selector that includes "Mirpur 12 (Default)", "Entire Dhaka", "Sylhet", "Rajshahi", and "Chittagong".
- **Pair D:** Create the central team repository (GitHub/Shared Drive). Write the introduction and problem statement of the final report, focusing on Mirpur 12's dense concrete profile and lack of green canopy.

Day 2: Gathering Satellite Imagery & Mocking "Ground" Data

- **Pair A:** Download the Landsat 8/9 or Sentinel-2 thermal and infrared bands for the Mirpur 12 region. Extract the base Land Surface Temperature (LST) and calculate the NDVI (Vegetation Index).
- **Pair B:** Build a JSON database model that accepts regional data parameters dynamically.
- **Pair C:** Integrate **Leaflet.js** into the web interface and hard-center the default map view directly over Mirpur 12 (`[23.8243, 90.3653]`).
- **Pair D:** Since field surveys are skipped, synthesize an internet-backed "ground verification" dataset. Generate a CSV file containing roughly 20 coordinate points distributed around Mirpur 12 (e.g., higher temperatures mapped near the Mirpur 12 Bus Stand/Metro Station concrete zones, and lower temperatures mapped near local residential pockets or DOHS borders).

Day 3: GIS Layers & API Delivery

- **Pair A:** Import the satellite data into QGIS/ArcGIS. Clip the raster data directly to the Mirpur 12 border, generate a base localized heatmap, and export it as an optimized layer format (like GeoJSON or structured coordinate arrays).
- **Pair B:** Write the logic to parse Pair A's GeoJSON/coordinate arrays and Pair D's mock CSV dataset into a single unified JSON response when `/api/heat-data?region=mirpur12` is queried.
- **Pair C:** Map out basic map-container styling and build custom markers/tooltips for Leaflet.js to handle data coordinates.
- **Pair D:** Compile the Phase 1 methodology details, including the satellite acquisition steps and coordinate tables, directly into the draft report.

Phase 2: Core Development & Analytical Modeling (Days 4–7)

Goal: Connect the data layers to the map interface and build your mathematical analysis for Mirpur 12.

Days 4–5: Interactive Heat Mapping & Algorithmic Correlation

- **Pair A:** Assist Pair C in correctly alignment-matching the spatial data bounds within the Leaflet window so the satellite heatmap overlays perfectly on top of the open-street base map.
- **Pair B:** Since deep machine learning is optional, write a clean, lightweight Python script using `scikit-learn` to calculate a simple Linear Regression model. Use it to map out the inverse relationship between vegetation density (NDVI) and localized heat retention:
$$\text{Predicted Temperature} = \alpha - (\beta \times \text{NDVI})$$
- **Pair C:** Implement `Leaflet.heat` or render custom styled polygons over Mirpur 12 that visually display high-heat zones (reds) vs cooler micro-climates (greens).
- **Pair D:** Draft the System Architecture block for the documentation. Draw a diagram mapping how data flows from the GIS/CSV sources, through the Python analytical script, to the web dashboard interface.

Days 6–7: Comparative Visualizations & Interface Polish

- **Pair A:** Verify that the calculated LST values line up cleanly with typical urban thermal expectations (e.g., concrete surfaces displaying several degrees higher than surrounding elements).
- **Pair B:** Build backend endpoints that supply summary statistics for Mirpur 12: peak hotspot location, average temperature, and an overall calculated thermal comfort score.
- **Pair C:** Integrate `Chart.js` or `Plotly` to generate bar and line charts on the dashboard side panel. Create visual comparisons showing Urban Built-up Surfaces vs Vegetated Areas within Mirpur 12.
- **Pair D:** Review the mathematical accuracy of the regression script output with Pair B and update the data analysis section of the report.

Phase 3: Regional Multi-City Expansion Loop (Days 8–9)

Goal: Scale up your system to include Entire Dhaka, Sylhet, Rajshahi, and Chittagong using your modular codebase.

Day 8: Rapid Multi-City Data Injection

- **Pair A:** Return to your GIS/Earth Engine environment. Rapidly download the broader, macro-level LST and NDVI averages for Entire Dhaka, Sylhet, Rajshahi, and Chittagong.
- **Pair B:** Re-run your regression script on the new regional datasets. Append these calculated city profiles into your backend JSON structure under unique tags (`dhaka_all`, `sylhet`, `rajshahi`, `chittagong`).
- **Pair C:** Update the frontend data call functions. Write an event listener so that when a user switches the selection dropdown, it triggers a dynamic UI function (e.g., `map.flyTo()`) to jump directly to the chosen city's center coordinates.
- **Pair D:** Collect geographic profile summaries for the new cities (e.g., noting Rajshahi's high summer heat indices or Sylhet's higher canopy ratios) to explain the distinct visual variances across the regions.

Day 9: Dynamic UI Binding

- **Pair A & B:** Ensure that the API accurately serves the correct spatial datasets instantly whenever a specific region token is requested by the frontend dashboard.
- **Pair C:** Bind the dynamic data calls to your Chart.js components. When switching cities, the map heat overlays, peak hotspot text, and comparison graphs must re-render to reflect that specific city's attributes instantly.
- **Pair D:** Expand the final report and slide deck formatting to include dedicated analysis sections for each newly added division.



Phase 4: System Integration, Testing & Presentation Prep (Days 10–12)

Goal: Eliminate system bugs, compile final documents, and run viva rehearsals.

Day 10: Complete System Validation & Bug Catching

- **Pairs A, B, & C (The Devs):** Run end-to-end integration testing. Thoroughly check for coordinate drifting on the map overlays, broken dropdown links, chart sizing issues, or server lag when swapping rapidly between Mirpur 12 and other cities.
- **Pair D:** Take high-resolution screenshots of every functional dashboard view (Mirpur 12, Entire Dhaka, Sylhet, Rajshahi, and Chittagong) and embed them into the slide presentations and report appendices.

Day 11: Document Finalization & Backup Demo Production

- **Pairs A, B, & C:** Assist Pair D by double-checking all technical terminologies and system architecture descriptions in the final files. Freeze code modifications once stable.
- **Pair D:** Finalize and polish the presentation PowerPoint slide deck. Record a clean, high-definition 3-minute video screen-capture walk-through of the fully functioning web application to use as an emergency presentation backup in case your evaluation room experiences internet connectivity drops.

Day 12: Full Team Viva Rehearsal Simulation

- **Entire Team (All 8 Members):** Conduct mock question-and-answer run-throughs. Every single team member—regardless of whether they focused on writing, backend, or GIS design—must be completely fluent in defining Urban Heat Islands, explaining why cities run hotter than rural domains, breaking down what an NDVI score means, and detailing the complete multi-city software workflow.